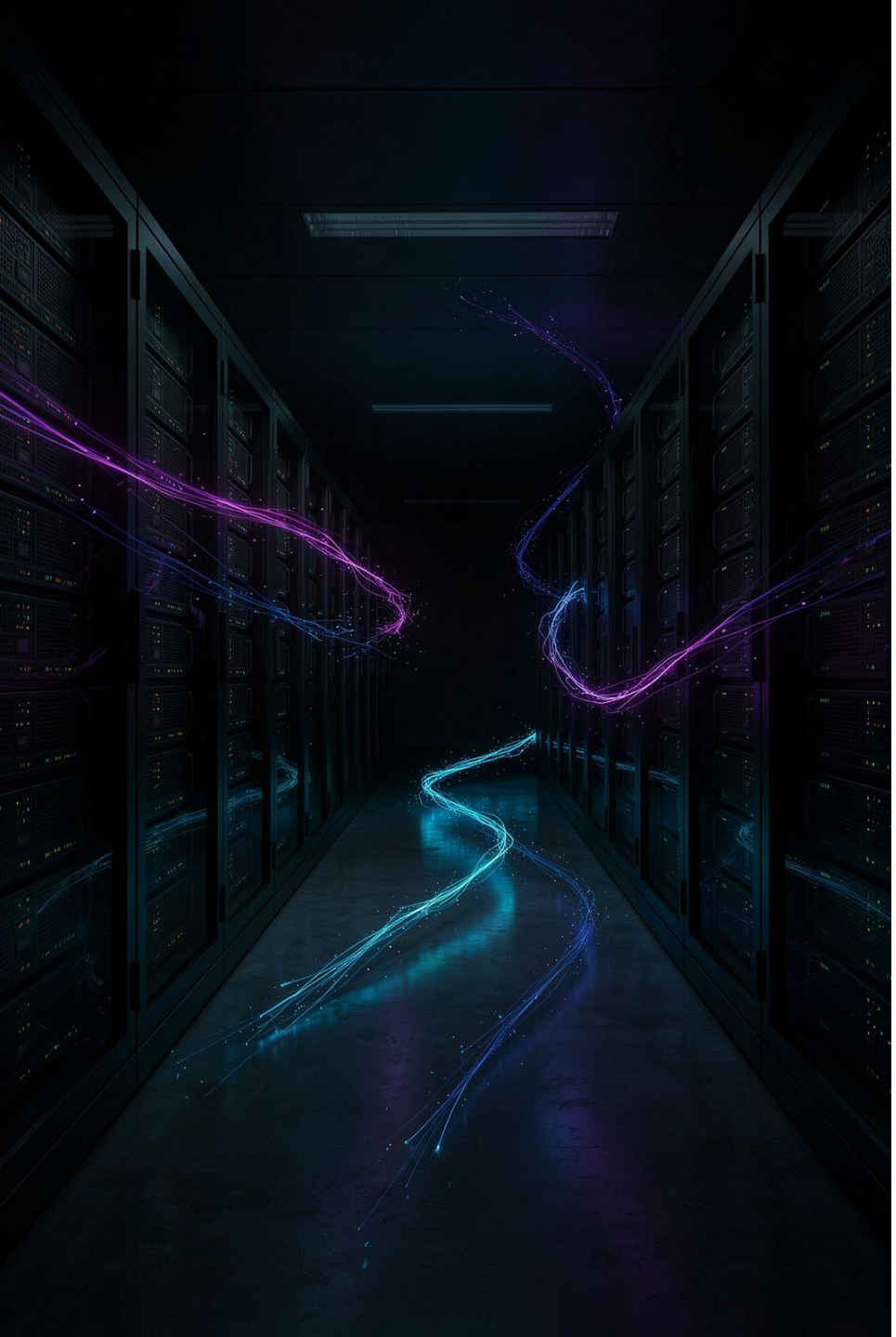


Module 2: Data Management for AIOps

Using GitHub as the backbone of your AI-powered operations pipeline — from raw data ingestion to version-controlled, production-ready workflows.

MODULE 2 OF 5

What We'll Cover in This Module

Think of AIOps like running a smart hospital. It needs constant data from multiple sources, a clean intake process, organized storage, and a reliable records system. This module walks through each of those layers using real-world analogies and GitHub tooling.

01

Data Sources

Logs, Metrics, Events & Traces

02

Collection & Aggregation

Gathering data from distributed systems

03

Cleaning & Storage

Preparing data for AI consumption

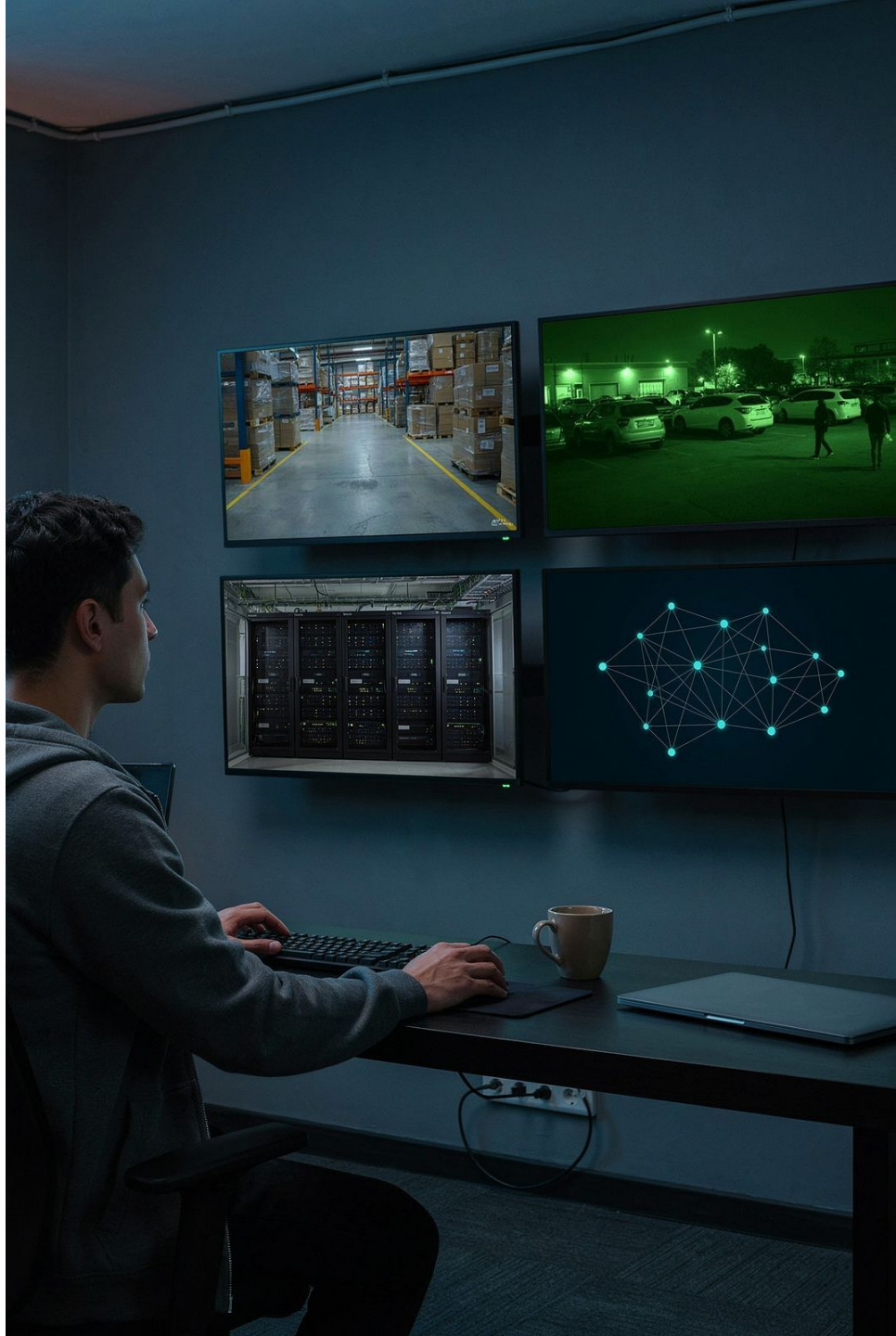
04

GitHub Management

Git LFS and GitHub Actions pipelines

The Four Pillars of AIOps Data

Just like a hospital monitors patients through vitals, cameras, alarms, and lab results — AIOps monitors systems through four distinct data types. Each tells a different part of the story.



Data Source #1 — Logs

Logs are the diary of your system. Every time something happens — a user logs in, a service crashes, a file is accessed — the system writes a timestamped entry into a log file.

📌 **Real-World Example:** Think of logs like the receipts a cash register prints. If a transaction fails, you scroll back through the receipts to find exactly where things went wrong.

- Application logs (errors, warnings, debug info)
- System logs (OS-level events)
- Security logs (auth attempts, access control)
- Audit logs (compliance and change tracking)

Sample Log Entry

[2024-03-15 14:32:01] ERROR: Database connection timeout after 30s — retrying (attempt 2/3)

[2024-03-15 14:32:31] WARN: High memory usage detected — 87% utilization

[2024-03-15 14:33:00] INFO: Service restarted successfully

Data Source #2 — Metrics

Metrics are **numerical measurements collected over time** — the pulse readings of your infrastructure. Unlike logs which describe events, metrics quantify system health continuously.



System Metrics

CPU usage, memory consumption, disk I/O, and network throughput. Like checking your car's RPM and fuel gauge while driving.



Application Metrics

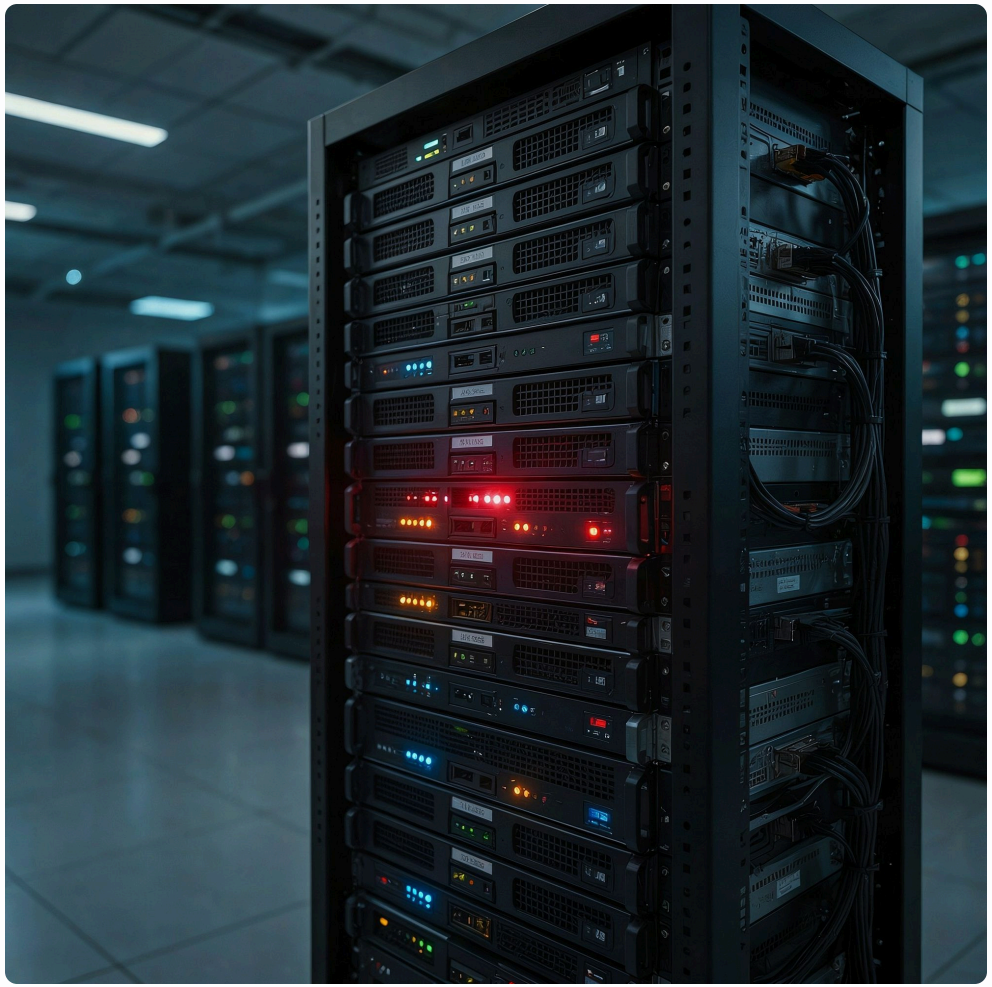
Request rates, error rates, response times (latency). Tools like Prometheus scrape these every few seconds.



Business Metrics

Active users, transactions per second, cart abandonment rate — connecting tech health to business outcomes.

Data Source #3 — Events



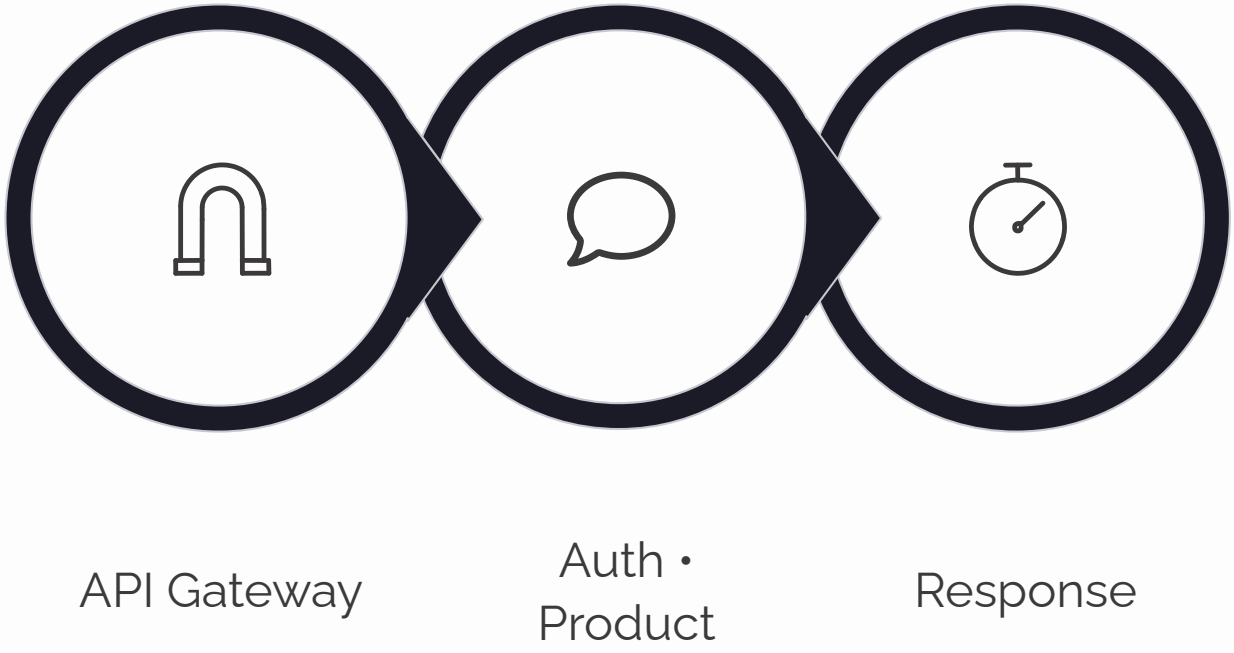
Events are significant state changes that demand attention. Unlike metrics (continuous) or logs (verbose), events are discrete triggers — something notable just happened.

📌 **Real-World Example:** A smoke detector doesn't constantly report "no smoke." It stays silent until it detects smoke — that's an event. Your system behaves the same way.

- Deployment events (new code pushed to production)
- Alert events (threshold breached — CPU > 90%)
- Scaling events (auto-scaler spun up 3 new instances)
- Failure events (service went down at 2:14 AM)

Data Source #4 — Traces

Traces follow a **single request as it travels through multiple services** in a distributed system. They answer the question: "Where did my request go, and where did it slow down?"



📌 **Real-World Example:** Imagine tracking a package from warehouse → truck → local hub → your door. A trace does the same for your API request across microservices — so you can pinpoint exactly which stop caused the delay.

Quick Recap — The Four Data Types

📋 Logs

Text records of events. *The system diary.*

📊 Metrics

Numeric measurements over time. *The vital signs.*

🔔 Events

Significant state changes. *The alarm bells.*

🔍 Traces

End-to-end request journeys. *The GPS trail.*

AIOps platforms like Dynatrace, Datadog, and New Relic ingest all four data types simultaneously to build a unified operational picture.

CHAPTER 2

Data Collection & Aggregation

With data coming from hundreds of services, servers, and containers — how do you collect it all without drowning in noise? This is where aggregation strategies come in.



How Data Gets Collected — The Pipeline

Data collection in AIOps follows a structured pipeline. Think of it like a city's water supply system — taps (sources) → pipes (collectors) → treatment plant (aggregator) → reservoir (storage).

1 Agents & Collectors

Lightweight software running on each server (e.g., Fluentd, Filebeat, OpenTelemetry agent) that reads and forwards data locally.

2 Message Brokers

Systems like Apache Kafka or AWS Kinesis act as buffers — absorbing high-volume data bursts without overwhelming downstream processors.

3 Aggregation Layer

Tools like Logstash or Fluentd aggregate, filter, and route data to the right destinations — databases, dashboards, or AI models.



Aggregation Methods in Practice

Not all data needs to be collected with the same frequency or strategy. Smart aggregation reduces cost and noise without losing signal.

Pull vs. Push

Pull: The aggregator asks each service for data on a schedule (Prometheus scrapes metrics every 15s). **Push:** Services send data to the collector as it happens (application logs forwarded to Elasticsearch in real-time).

Sampling

For traces with massive volume, only capture 1 in every 100 requests (1% sampling). Still statistically meaningful, but 100x cheaper. Netflix uses adaptive sampling for this reason.

Batching & Streaming

Batching collects data periodically (hourly log uploads to S3). Streaming processes data in real-time (Kafka streams for live anomaly detection). Most AIOps platforms use both.



CHAPTER 3

Data Cleaning, Preparation & Storage

Raw data is like unprocessed ore — valuable, but not yet usable. Before AI models can learn from it, the data must be cleaned, structured, and stored intelligently.

Why Data Cleaning Matters

Garbage in, garbage out. If your AIOps model is trained on noisy, duplicate, or incomplete data, it will make poor predictions — like a doctor diagnosing based on a broken thermometer.

Common data quality problems in AIOps:

- **Duplicate logs** — same error reported 500 times in 1 second due to a retry loop
- **Missing values** — metrics with gaps because an agent went offline
- **Inconsistent formats** — timestamps in 3 different timezone formats across services
- **Noise** — routine "heartbeat" messages flooding logs and hiding real errors

Cleaning Techniques

- 🔁 **Deduplication** — collapse 500 identical log lines into one with a count
- 📅 **Normalization** — standardize all timestamps to UTC ISO 8601
- 🔕 **Noise filtering** — drop routine health-check pings below a severity threshold
- 🩹 **Imputation** — fill metric gaps with interpolated values or flag as missing
- 🏷️ **Enrichment** — add metadata like service name, region, and team owner to each record

Data Storage Strategies

Different data types have different access patterns — and choosing the right storage layer saves both cost and query time. Think of it like choosing between a filing cabinet, a bookshelf, and a cold archive.



Hot Storage

Recent data (last 7–30 days) kept in fast, queryable systems like Elasticsearch or ClickHouse. Accessed constantly for real-time dashboards and alerts.



Warm Storage

Data from the past 30–90 days. Stored in columnar databases like Apache Parquet on S3. Slower to query but much cheaper — used for trend analysis.



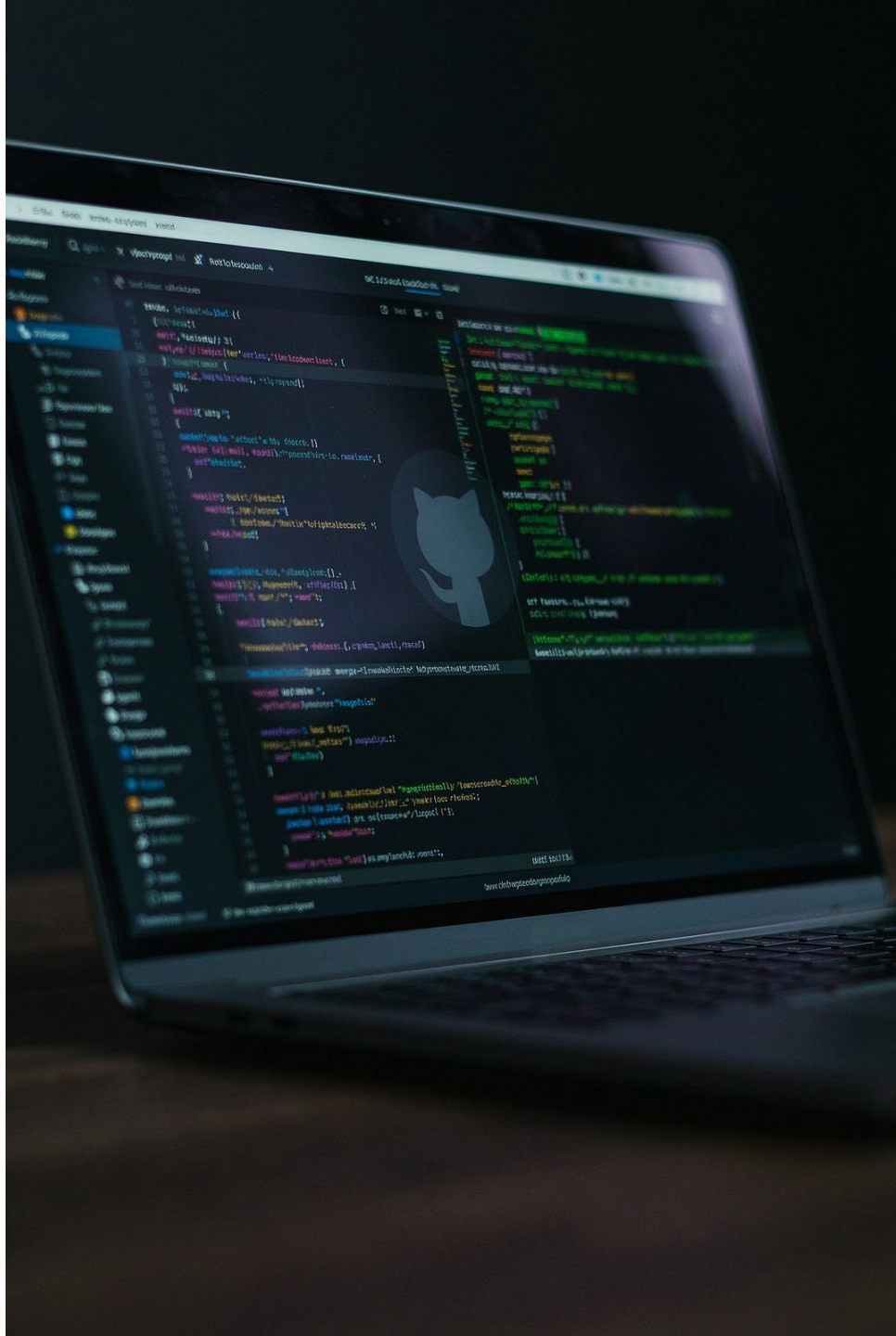
Cold Storage

Historical data (90+ days) archived to object storage like AWS Glacier or GCS Nearline. Extremely cheap, rarely accessed — used for compliance and model retraining.

CHAPTER 4

GitHub for Data Version Control

Code isn't the only thing that needs versioning. In AIOps, the data pipelines, configuration files, and even datasets themselves must be tracked, versioned, and collaborated on — just like source code.



The Problem: Git Wasn't Built for Big Data

Standard Git struggles with large binary files. If your team commits a 2GB training dataset directly to a repository, Git copies the entire file into history on every change — your repo balloons to 20GB in weeks.

📌 **Real-World Example:** Imagine photocopying an entire encyclopedia every time someone corrects a single typo, and storing every copy in a filing cabinet. That's what happens to your Git repo without LFS.

Git LFS — The Solution

Git Large File Storage (LFS) replaces large files in your repository with tiny text pointers, while storing the actual file content on a remote LFS server.

- ✅ Repo stays small and fast
- ✅ Large files are versioned and trackable
- ✅ Only download large files when you need them
- ✅ Works with existing Git workflows

Using Git LFS — Step by Step

Setting up Git LFS is straightforward. Here's how a data engineering team would configure it to manage AIOps training datasets and model artifacts.

1

Install LFS

Run `git lfs install` once on your machine to initialize LFS for your Git installation globally.

2

Track File Types

Tell LFS which files to manage: `git lfs track "*.csv" "*.parquet" "*.h5"` — this creates a `.gitattributes` file.

3

Commit Normally

Use standard `git add`, `git commit`, and `git push` commands. LFS handles the heavy lifting automatically behind the scenes.

4

Pull Large Files

When a teammate clones the repo, large files download on demand with `git lfs pull` — keeping their clone lightweight by default.

GitHub Actions for Data Pipelines

GitHub Actions lets you automate your entire data pipeline as code — triggered by events like a new data file being pushed, a schedule (daily at midnight), or a pull request review.

📌 **Real-World Example:** Think of GitHub Actions as an assembly line robot. When a new part arrives (data file pushed), the robot automatically inspects it (validation), cleans it (preprocessing script), and places it in the right bin (pushes to storage). No human intervention needed.

Common AIOps pipeline triggers:

- New raw log file pushed → trigger cleaning workflow
- Every night at 2 AM → retrain anomaly detection model
- Pull request opened → validate data schema changes



Sample GitHub Actions Workflow

Here's a real example of a `.github/workflows/data-pipeline.yml` file that automatically validates and preprocesses new data every time it's pushed to the `main` branch.

```yaml
name: AIOps Data Pipeline

on:
  push:
    branches: [main]
    paths: ['data/raw/**']
  schedule:
    - cron: '0 2 * * *'   # Runs daily at 2 AM

jobs:
  validate-and-clean:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3
        with:
          lfs: true        # Pull LFS files too

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install pandas great-expectations

      - name: Validate data schema
        run: python scripts/validate_data.py

      - name: Run cleaning pipeline
        run: python scripts/clean_and_store.py

      - name: Upload to data warehouse
        run: python scripts/upload_to_storage.py
```

This single workflow file replaces hours of manual data processing — and every change to the pipeline is tracked in Git history.



Module 2 — Key Takeaways

📋 Know Your Sources

Logs, Metrics, Events, and Traces each serve a unique role. Use all four for complete observability.

🔗 Collect Smartly

Use agents, brokers, and aggregators. Apply sampling and batching to balance cost vs. insight.

🧹 Clean Before AI

Deduplicate, normalize, and enrich data. Store in hot/warm/cold tiers based on access needs.

🐙 GitHub as Control Tower

Use Git LFS for large datasets and GitHub Actions to automate your entire data pipeline as code.